Many Ruby programs store and later call blocks in this way—it's a great way of implementing callbacks, dispatch tables, and so on. But you can go one step further. If a block can be turned into an object by adding an ampersand parameter to a method, what happens if that method then returns the Proc object to the caller? What can you do with that Proc object?

Well, you can call it, for one thing…

```
def create_block_object(&block)
  block
end

bo = create_block_object { |param| puts "You called me with #{param}" }

bo.call(99)
bo.call("cat")
```

produces:

```
You called me with 99
You called me with cat
```

The create_block_object method converts its block argument to the variable named block and then returns it. The returned value is a Proc object and can be called with the call method.

Creating a variable with a block value is so useful that Ruby provides multiple ways to do so:

The one you might see the most in newer code is the "stabby lambda" syntax, where the -> operator declares that a block is coming:

```
bo = ->(param) { puts "You called me with #{param}" }
bo.call(99)
bo.call("cat")
```

produces:

```
You called me with 99
You called me with cat
```

The stabby lambda is a shortcut for the Ruby Kernel method lambda:

```
bo = lambda { |param| puts "You called me with #{param}" }
bo.call 99
bo.call "cat"
```

produces:

```
You called me with 99
You called me with cat
```

There's a related Kernel method called proc:

```
bo = proc { |param| puts "You called me with #{param}" }
bo.call 99
bo.call "cat"
```

produces:

```
You called me with 99
You called me with cat
```

Both proc and lambda invoke the new method of the Proc class, although current style prefers one of the above mechanisms to using Proc.new directly: